MODULE NAME:	MODULE CODE:
PROGRAMMING 3B	PROG7312/w
ADVANCED APPLICATION DEVELOPMENT	AAPD7112/w

ASSESSMENT TYPE: POE (PAPER & MARKING RUBRICS)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A minimum of 45 HOURS is suggested to complete this assessment.

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity and Property Rights Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. ***Please ensure that you submit your assignment through Turnitin. Please make sure you attach a similarity report to your POE if you are required to submit a hard copy of your PoE.***
3. ***Make a copy of your assignment before handing it in.***
4. *Assignments must be typed unless otherwise specified.*
5. *Begin each section on a new page.*
6. *Follow all instructions on the PoE cover sheet.*
7. *This is an individual assignment.*

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate.** The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage.**
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage.**
- **Where both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: **Consistent and congruent referencing**	**Minor errors** **Deduct 5% from overall percentage.** **Example: if the response receives 70%, deduct 5%. The final mark is 65%.**	**Major errors** **Deduct 10% from the overall percentage.** **Example: if the response receives 70%, deduct 10%. The final mark is 60%.**
Consistency The **correct referencing style** for the **discipline** – i.e., either **Harvard**, OR **APA** (for Psychology), OR **Law**, OR **IEEE** (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. **Concepts** and **ideas** that are **quoted** and/or **paraphrased** are referenced consistently throughout. **Position of the in-text reference**: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	**Minor inconsistencies:** • The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. • For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. • Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. • Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	**Major inconsistencies:** • Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. • Multiple referencing styles for the same source types have been used. • For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. • Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. • Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency • **Each source** reflected within **in-text references** is included accurately in the **bibliography/reference list.** • All **bibliography/reference list** entries are in the **required order** for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). • All **direct quotes** and **paraphrases** have been **integrated appropriately** into the text using introductory phrases, accurate grammar, etc.	**Minor incongruences:** • There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). • There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. • An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	**Major incongruences:** • No relationship/several incongruencies between the in-text referencing and the bibliography/reference list. • For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. • Most sources are listed in a haphazard order throughout the bibliography/reference list. • Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

Portfolio of Evidence (PoE) — Background: The Smart-X IoT Mesh Ecosystem

Scenario

With the explosive growth of edge computing and automation, a local South African technology venture is developing **Smart-X**, a hybrid Internet of Things (IoT) ecosystem designed to monitor and manage distributed environments (such as hydroponic farms, automated real estate utility trackers, and smart grid installations).

The infrastructure relies on thousands of microcontrollers (like ESP32 chips) publishing constant, multi-typed telemetry data (floats for soil moisture, integers for power wattage, booleans for valve states) back to a central gateway. To process, validate, and optimise this high-throughput data stream, the engineering team requires an advanced application suite.

NB: Please make sure to do full research on the topic on IOT, because marks will not be awarded to a general application

Note on Hardware and Data Seeding:

For the primary purposes of this assessment, this environment is a simulation. You will be expected to heavily seed your application with mock telemetry data to prove your data structures work efficiently under load. However, you are more than welcome to include real-world devices and live sensor data if you have access to the physical hardware!

Architecture Flexibility and Freedom of Choice

Unlike rigid legacy systems, the architecture must be modernised. You are **not restricted to standard desktop applications**. Instead, you have the architectural freedom to choose their own frontend tech stack while leveraging **.NET 10** as the backend backbone.

You may choose to build your solution using any of the following combinations:

- **Web-First:** An ASP.NET Core Minimal API backend paired with a Blazor WebAssembly, React, or Vue frontend.
- **Cross-Platform Mobile/Desktop:** A .NET MAUI or Avalonia UI application interacting with a self-hosted local Web API.
- **Traditional Desktop:** A WPF or Windows Forms UI communicating seamlessly with an internal or external processing engine.

Instructions

Complete the tasks below to establish the core data ingestion engine of the Smart-X ecosystem.

Part 1 — Smart-X Data Ingestion and Validation Gateway (Marks: 100)

Learning Units: LU1 – LU2

This part has two tasks – **Research** (20 marks) and **Implementation** (80 marks).

Task 1: RESEARCH (Marks: 20)

High-throughput IoT systems suffer significantly from user drop-off if the configuration and debugging consoles are obtuse or overly complex. Conduct online research on **developer-centric user engagement strategies** (e.g. real-time visual telemetry feedback, proactive alert gamification, or simplified onboarding flows) suitable for a technical management interface. Make sure to use journal articles.

In a Word document, write a report providing:

1. **List five user engagement or dashboard interactivity strategies** considered during your research to make telemetry data clear and actionable.
2. **Provide a 500-word explanation** and justification of your chosen dashboard engagement strategy, specifically evaluating how it helps users easily spot and troubleshoot anomalous data spikes or sensor disconnects.

Formatting requirements:

- 1.5-line spacing. Arial or Times New Roman font (11 or 12-point).
- Follow proper academic referencing standards.
- In a report format with max 2-3 pages
- Include diagrams to help explain any concepts

Remember to reference the sources used.

Task 2: IMPLEMENTATION (.NET Architecture and API Integration) (Marks: 80)

You are tasked with building the ingestion API and client-facing configuration dashboard for the Smart-X system. The application must process varied sensor data types efficiently without type-coercion overhead.

Functional Requirements

- **Startup Interface:** The application gateway must display a landing page or menu offering three architectural pillars:
  - **Sensor Data Ingestion and Telemetry** (To be implemented in this part).
  - **Real-Time Command Stream and History** (Disabled – to be implemented in Part 2).
  - **Network Topology and Mesh Routing** (Disabled – to be implemented in the final PoE).
- **API Integration Layer:** Implement a .NET-based Web API (e.g. ASP.NET Core Minimal APIs or Controllers) to act as the primary receiver of telemetry data. The frontend dashboard must communicate with this API layer to push and retrieve data.
- **Sensor Payload Management:** Users must be able to simulate or attach custom sensor registration records, including:
  - Device MAC Address / Unique Identifier.
  - Sensor Deployment Location (e.g. Room, Zone, Node ID).
  - Sensor Category Selection (e.g. Environmental, Power Consumption, Actuator).
- **Media/Log Attachment:** Allow users to attach physical device configuration files, deployment photos, or hardware log files directly to a specific sensor profile via an optimised file upload mechanism (OpenFileDialog or an API-based multipart file uploader).
- **Dynamic Engagement Feature:** Integrate your chosen research engagement strategy. *Remember to be unique, not just a progress bar.*

Technical and Language Requirements

To ensure optimal performance on resource-constrained gateway devices, you must explicitly integrate the following advanced object-oriented C# concepts:

- **Generics:** Design a reusable generic wrapper class, TelemetryPacket<T>, to handle disparate incoming data structures uniformly (e.g. passing a float for temperature, an int for power metrics, or a bool for smart switch triggers) without resorting to expensive boxing/unboxing operations.

- **Operator Overloading:** Overload operators (such as +, -, or logical comparison operators) on your sensor data structures to allow direct aggregation or delta comparisons of sensor values in code (e.g. calculating the aggregate load of two smart meters: Meter3 = Meter1 + Meter2).
- **Advanced Arrays and Lists:** Utilise multi-dimensional or jagged arrays to manage sequential historical batches of raw telemetry arrays before transferring them into optimised Collections (List<T>).
- **Recursion:** Write a recursive validation algorithm to parse nested device deployment trees or multi-tier configuration profiles (e.g. validating that a node is safely configured within Sub-Zone B -> Zone 1 -> Facility A).

Note: If the code does not **compile** and **run**, across both the frontend client and backend API layers, no marks will be awarded for any application functionality.

Deliverables for Part 1

- A Word document containing your research paper.
- Full source code directory containing the API and client application. All stored on GitHub. A 5% penalty if no GitHub is used with an appropriate number of commits. Each project should be over 20 commits at least.
- Include any Docker files if included (FYI, it is suggested to use docker like you did in PROG7311)
- A comprehensive README.md file providing explicit setup instructions on how to restore dependencies, compile the project, boot the backend API, and run the client user interface.
- YouTube Video demo or Presentation, your lecture will advise on this. Please consult the lecturer of the module on your campus.

Important! You will build on this application in Part 2 and the PoE. So, keep a copy of your code in a safe place!

[Total MARKS: 100]

Part 2 — Smart-X Real-Time Processing and Device Management (Marks: 100)

Learning Units: LU1 – LU4

Introduction:

In Part 2, the Smart-X IoT ecosystem evolves from a static data ingestion gateway into a dynamic, real-time processing engine. Managing a live network of ESP32 microcontrollers, smart plugs, and environmental sensors requires highly optimised data handling to prevent bottlenecks. Remember, you can simulate all this data, but it has to hit the API like it was a device. You will implement advanced data structures to manage incoming telemetry queues, store device states, and predict system anomalies.

Develop the Application Logic and UI Integration

Main Menu and Navigation (30 Marks)

- Update your frontend application (Web, MAUI, or Desktop) with an organised navigation menu presenting the following active modules:
  - Sensor Data Ingestion and Telemetry (Implemented in Part 1).
  - Real-Time Command Stream and History (To be implemented in this part).
  - Network Topology and Mesh Routing (To be implemented in the final PoE).
- Ensure seamless, asynchronous navigation between these views without losing application state or crashing the API backend.

Real-Time Command Stream and History Page (70 Marks)

- Upon accessing the “Real-Time Command Stream,” present a highly responsive dashboard that visualises incoming sensor data and allows the user to issue manual override commands (e.g. triggering a water pump, toggling a smart relay).
- Implement a search or filter functionality allowing users to efficiently find specific devices based on their operational categories or active alert states.
- Utilise advanced data structures to optimise how the API backend processes and stores this volatile data.

Technical Requirements for Real-Time Command Stream (40 Marks)

Mark allocation breakdown with suggestions of data structures that could be used:

Stacks, Queues, Priority Queues (15 Marks)

- **Message Queues:** Implement a Queue<T> to handle the standard First-In-First-Out (FIFO) processing of incoming telemetry packets hitting the API.
- **Priority Queues:** Implement a Priority Queue for critical alerts. For example, a severe power spike or a critical moisture drop in a hydroponics node must bypass the standard queue and be processed immediately.
- **Stacks:** Implement a Stack<T> to manage a history of manual override commands sent to devices. This will serve as an "Undo" feature, allowing operators to quickly revert the last issued command (e.g. reverting a "shut down all valves" action).

Hash Tables, Dictionaries, Sorted Dictionaries (15 Marks)

- **Dictionaries:** Utilise a Dictionary<TKey, TValue> to store the live, registered devices. Use the device MAC address or a unique node identifier as the key for instantaneous O(1) lookups when a new telemetry packet arrives.
- **Sorted Dictionaries:** Use a Sorted Dictionary to organise and display historical sensor logs sequentially by timestamp, ensuring the frontend dashboard can render timeline graphs efficiently.

Sets (10 Marks)

- **Hash Sets:** Incorporate a HashSet<T> to maintain a list of currently active, unique error states or disconnected nodes. Because sets only store unique elements, this efficiently prevents the system from processing duplicate disconnect alerts for the same sensor.

Predictive Action and Recommendation Engine (30 Marks)

Instead of standard event recommendations, implement an **Automated Action Engine** based on historical sensor queries and data patterns.

- **Pattern Analysis:** Analyse the history of user searches or manually triggered commands (e.g. if a user frequently searches for "Pump A" right after "Moisture Sensor B" drops below 20%).
- **Algorithmic Suggestion:** Use an appropriate algorithm or data structure to proactively suggest related commands or highlight specific potentially problematic devices (faults, out of range readings, etc.) on the dashboard before the user searches for them.
- **Presentation:** Present these predictive recommendations clearly in the UI as "Suggested Actions" or "Automated Insights."

Note: If the code does not **compile** and **run** across both your frontend and API layers, no marks will be awarded for any application functionality.

Deliverables for Part 2

- Full source code directory containing the API and client application. All stored on GitHub. A 5% penalty if no GitHub is used with an appropriate number of commits. Each project should be over 20 commits at least.
- Include any Docker files if included (It is suggested to use Docker and/or Docker Compose like you did in PROG7311).
- An updated README.md file with explicit instructions on how to test the new queue and dictionary implementations (e.g. how to simulate a high-priority alert).
- YouTube video demo or presentation, your lecture will advise on this. Please consult the lecturer of the module on your campus.

PoE — Smart-X Network Topology and Mesh Routing (Full Functioning App) (Marks: 100)

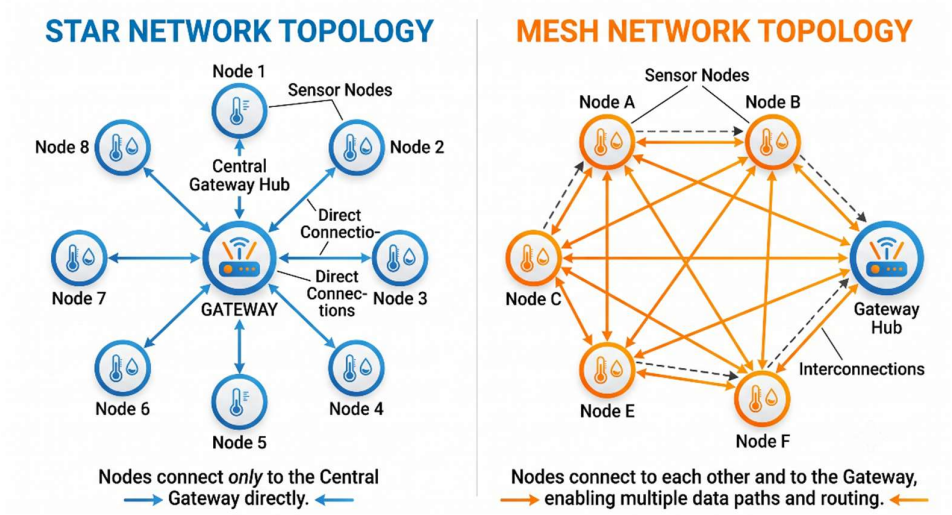
Learning Units: All Learning Units

Introduction:

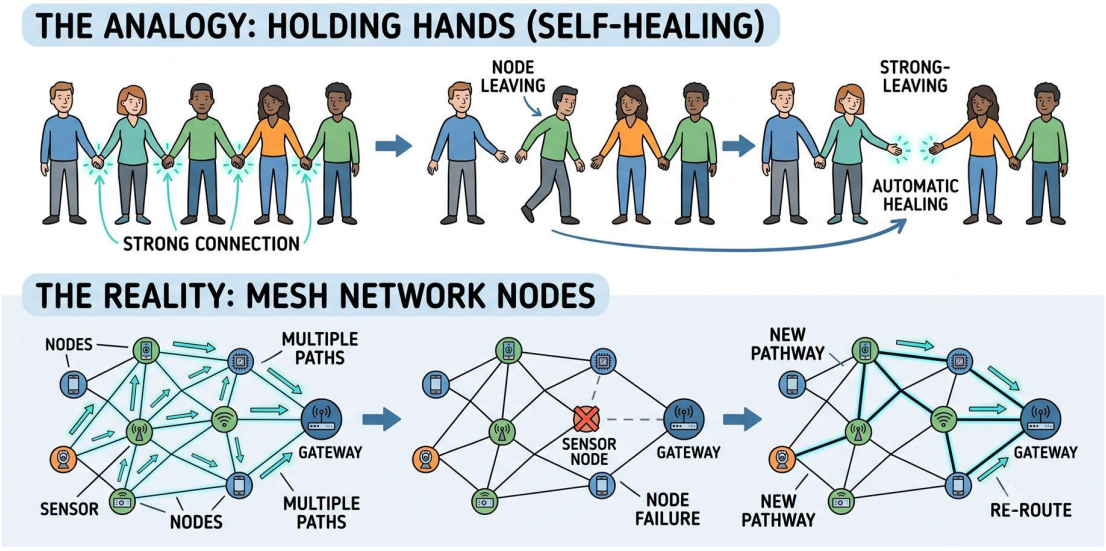
In this final phase of the project, you will finalise the Smart-X ecosystem. Managing thousands of dispersed IoT nodes requires an efficient way to visualise the network topology, monitor device health, and optimise how data packets route back to the central gateway. You will integrate advanced data structures including Trees, Heaps, and Graphs to finalise the application.

Understanding Mesh Network Theory

To successfully implement these data structures, it is vital to understand the physical architecture you are modelling: a Mesh Network. Unlike standard Wi-Fi networks where every single device must connect directly to a central router (a Star Topology), a mesh network allows each IoT node to connect to multiple neighbouring nodes. In this setup, data packets can "hop" from sensor to sensor until they finally reach the central gateway.



This decentralisation is crucial for large-scale hardware deployments. If a single node goes offline due to a hardware failure or power loss, the network automatically "heals" itself by finding a new path for the data to travel. This exact concept is the backbone of modern smart home and industrial IoT protocols that you will encounter in the field, such as Zigbee, Thread, and Espressif's ESP-NOW.



A MESH NETWORK "HEALS" ITSELF JUST LIKE PEOPLE ADJUSTING THEIR HANDSHAKES TO KEEP THE CONNECTION STRONG.

By utilising Graphs and Minimum Spanning Trees (MST) in this task, you are mathematically replicating how these real-world protocols determine the fastest, most power-efficient routes for their data packets to travel.

Final Application Logic and UI Integration

Main Menu Integration

- Update your frontend application to ensure all menu options are fully active and interconnected:
  - Sensor Data Ingestion and Telemetry (Implemented in Part 1).
  - Real-Time Command Stream and History (Implemented in Part 2).
  - **Network Topology and Mesh Routing** (To be implemented in this task).

Network Topology and Mesh Routing Page

- Create a dedicated dashboard view that visualises the active IoT mesh network.
- Allow users to track the routing paths of specific sensor nodes using their unique MAC addresses or identifiers.
- Display a well-organised historical log of a selected sensor's telemetry data.

<u>Technical Requirements (50 Marks)</u>

Mark allocation breakdown with suggestions of data structures that could be used:

Basic Trees, Binary Trees, Binary Search Trees, AVL Trees, Red-Black Trees (20 Marks)

- **Historical Data Querying:** Implement a Binary Search Tree, AVL Tree, or Red-Black Tree to store historical, timestamped telemetry data for the sensors.
- This structure must be used to efficiently query and retrieve specific data ranges (e.g. pulling all moisture readings between 14:00 and 16:00 for Node A) without iterating through a massive, unorganised list.

Heaps, Graphs, Graph Traversal, Minimum Spanning Tree (30 Marks)

- **Network Graphing:** Model the physical IoT mesh network using a Graph structure, where vertices (nodes) represent the ESP32 sensors, and edges represent the wireless connections between them.
- **Graph Traversal:** Implement Breadth-First Search (BFS) or Depth-First Search (DFS) to allow the system to identify all devices currently connected to a specific subnet or gateway, and identify which ones are down/where to hop in that case?
- **Minimum Spanning Tree (MST):** Implement Prim's or Kruskal's algorithm to calculate the Minimum Spanning Tree of the network. This must demonstrate the most power-efficient routing path for telemetry data to hop through the mesh nodes to reach the central server.

Documentation and Reporting (50 Marks)

Implementation Report (20 Marks)

- **README File:** Compile a detailed README.md file explaining how to compile, run, and use the complete application suite.
- **Data Structure Explanations:** For each implemented data structure from this task (Trees, Graphs, MST), provide an in-depth explanation of its specific role and contribution to the efficiency of the "Network Topology" feature, including relevant code examples.

Project Completion Report (20 Marks)

- Write a comprehensive report detailing the completion of the entire Smart-X project.
- Discuss the specific architectural or algorithmic challenges faced during the implementation of Task 3 (e.g. integrating the C# backend with your chosen frontend) and how they were overcome.
- Share insights into the key learnings acquired throughout the project, including new skills, problem-solving approaches, and programming techniques, making sure to link back to the actual task/assignment, and not just a generic answer.

Technology Recommendations (10 Marks)

- Suggest additional technologies, APIs, or tools that could enhance the functionality, scalability, or performance of the Smart-X ecosystem (e.g. integrating Google Cloud services for distributed database hosting, utilising a CI/CD pipeline via GitHub Actions, or embedding an LLM API to summarise daily telemetry alerts, https://www.home-assistant.io/).
- Justify these recommendations based on potential benefits and compatibility with the .NET project ecosystem.

Note: Submit a file listing the updates you have made based on feedback from Parts 1 and 2.

Note: If the code does not **compile** and **run**, no marks will be awarded for any application functionality.

Submit the following items for this part:

1. A **PDF document** containing the report.
2. **Source code** for the application, which must include the **complete code of the functioning application**. All stored on GitHub. A 5% penalty if no GitHub is used with an appropriate number of commits. Each project should be over 20 commits at least.
3. Include any Docker and/or Docker Compose files if included (FYI, it is suggested to use docker like you did in PROG7311).
4. A comprehensive README.md file providing explicit setup instructions on how to restore dependencies, compile the project, boot the backend API, and run the client user interface.
5. YouTube video demo or presentation, your lecture will advise on this. Please consult the lecturer of the module on your campus.
6. A file listing the **updates** that you have made based on **feedback** from your lecturer.

Appendix A - PoE Marking Rubrics

Assessment Sheet (Marking Rubric)

Please note: Tear off this section and **attach** it to your work when you submit it/ If this is an online submission, then this information needs to be included in the online submission.

MODULE NAME:	**MODULE CODE:**
PROGRAMMING 3B	**PROG7312/w**
ADVANCED APPLICATION DEVELOPMENT	**AAPD7112/w**

STUDENT NAME:
STUDENT NUMBER:

PART 1 -Task 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Identification of IoT Dashboard User Engagement Strategies [5 Marks]	• No user engagement strategies are listed, or they are entirely unrelated to software or dashboards.	• Only 1 or 2 dashboard engagement strategies are listed, with limited relevance to tracking telemetry or IoT configurations.	• 3 or 4 relevant strategies are listed, demonstrating a solid understanding of user dashboard design.	• 5 highly relevant, well-defined dashboard engagement strategies are listed, specifically tailored to technical IoT systems.	
	0 -1Mark	**2-3 Marks**	**4 Marks**	**5 Marks**	

Research:	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
Explanation ,planning and Justification of Chosen Strategy **[10 Marks]**	• No explanation or justification is provided, or the logic is entirely flawed	• Some details are provided, but the explanation lacks depth or fails to explain how it helps users catch telemetry anomalies or sensor dropouts.	• A clear 500-word explanation and planning is provided, outlining the chosen strategy and justifying its use for analysing real-time data flows.	• An exceptional 500-word explanation and planning is provided, seamlessly connecting the engagement strategy to minimising troubleshooting times for complex IoT systems.	
	0 – 3 Marks	**4 – 6 Marks**	**7 – 8 Marks**	**9 – 10 Marks**	

Referencing and Citations **[5 Marks]**	• No proper referencing is provided.	• Referencing is present but lacks accuracy or proper citation format.	• References are mostly accurate, with minor issues in citation format.	• Proper referencing and citations are used, following the given article and other relevant sources.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	

PART 1 -Task 2					
	Does not meet the required standard	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Gateway Startup Layout and API Integration making use of the data structure **[10 Marks]**	• Startup menu is non-functional or missing entirely. No connection to a .NET API backend.	• Gateway layout is present, but communication with the .NET API features severe routing bugs or synchronous blocking.	• Clean architectural menu. The application communicates successfully with the .NET API backend with minor inefficiencies such as incorrect implementation of the connection string.	• Flawless startup experience. Asynchronous .NET API communication is clean, highly responsive, and robust.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8 Marks**	**9 - 10 Marks**	
PART 1 -Task 2					
App Functionality: **Sensor Telemetry Data Ingestion and UI** **[10 Marks]**	• The sensor registration or data ingestion page is completely broken or omitted.	• Ingestion page is present, but input forms (MAC address, zone location, category) contain notable data validation bugs.	• Ingestion functionality is solid, allowing successful submission of various device records with minor UI quirks and minor validation.	• Exceptional ingestion module. Robust client-side validation paired with immediate, accurate backend API parsing.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8 Marks**	**9 - 10 Marks**	

PART 1 -Task 2					
Advanced OOP: Generics and Overloading **[10 Marks]**	• TelemetryPacket<T> generic wrapper or operator overloading is missing.	• Implemented but flawed; relies on expensive boxing/unboxing or implements overloading without structural purpose.	• Generics are used correctly to handle disparate structures; operator overloading successfully aggregates sensor values.	• Brilliant application of generic type parameters and creative operator overloading for direct, high-performance data manipulation.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8 Marks**	**9 - 10 Marks**	
PART 1 -Task 2					
Data Handling: Arrays and Recursion **[10 Marks]**	• Jagged/multi-dimensional arrays or recursion are absent or cause application stack overflows.	• Data structures are present but used poorly; recursive configuration validation lacks a clear base case or breaks under depth.	• Sequential historical telemetry data is successfully stored in jagged/multi-dimensional arrays; recursive node validation works.	• Masterful optimisation of memory layouts; recursive validation algorithms are elegant, perfectly handling deep network hierarchies.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8 Marks**	**9 - 10 Marks**	

PART 1 -Task 2					
Media / Log File Upload Attachment [10 Marks]	• File attachment feature is completely non-functional or omitted.	• File uploader is present but exhibits notable bugs when saving configs or uploading images to the backend API.	• Users can attach logs or photos to sensor profiles via an optimised file uploader with minor layout flaws.	• Flawless media/log upload mechanism handling multipart streams efficiently without degrading backend performance and full encryption included.	
	0 – 3 Marks	**4 – 6 Marks**	**7 – 8 Marks**	**9 – 10 Marks**	
PART 1 -Task 2					
Dynamic Dashboard Engagement Integration [10 Marks]	• A list is not used at all to store user-reported issues.	• Named strategy is present but non-dynamic, or bugs impact its effectiveness (e.g. progress bars don't update dynamically).	• Engagement strategy is integrated well, dynamically reflecting real-time system states or notifications to the user.	• Engagement strategy is integrated seamlessly, dramatically enhancing dashboard readability and system interactivity.	
	0 – 3 Marks	**4 – 6 Marks**	**7 – 8 Marks**	**9 – 10 Marks**	

PART 1 -Task 2					
User Interface Design Considerations [10 Marks]	• The application layout is visually broken, lacks clear typography, and fails to handle resizing.	• UI has major inconsistencies in colour schemes, lacks descriptive labels, or has slow feedback loops.	• Interface is mostly consistent and clear, accommodating standard resolutions with clean feedback alerts.	• Visually outstanding, intuitive UI design that scales fluidly across device sizes, keeping users informed via crisp feedback loops.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8 Marks**	**9 - 10 Marks**	
PART 1 -Task 2					
Collections Optimisation and Lists [5 Marks]	• Generic Collections or Lists are not used to organise ingestion pipelines.	• Lists are used but combined haphazardly with legacy data structures, resulting in severe processing bottlenecks.	• System effectively shifts data from raw arrays into generic List<T> formats with minor inefficiencies.	• Optimal, consistent usage of collections throughout the pipeline to manage and track ingestion states elegantly. The collection is a custom data structure.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	

PART 1 -Task 2					
Documentation and Readme Quality [5 Marks]	• No readme file is submitted.	• README contains minimal info, making it exceptionally difficult to configure, restore dependencies, or run the multi-layered stack. Little to no Markdown included	• Sufficient readme provided outlining steps to launch the API and client frontend, though missing edge-case details	• Exceptional README.md, live or video demo giving precise, step-by-step commands to restore, build, boot the .NET API, and deploy the client app.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	
Incorrect GitHub used or not GitHub Used -5 Marks	• No GitHub repository submitted.	• GitHub repository submitted with limited commit history and limited evidence of collaboration. • No README.md file included.	• GitHub repository submitted with good commit history (25 commits). • Evidence of collaboration (multiple contributors). • README.md file included but not formatted in Markdown.	• GitHub repository submitted with excellent commit history (25+ commits and well-structured commit messages). • Evidence of collaboration (multiple contributors). • README.md file included and perfectly formatted in Markdown.	
	-5 Marks	**-4 or -3 Marks**	**-2 or -1 Marks**	**No deduction**	
PART 1 TOTAL					**/100**

Notes to Students:

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Main Menu Navigation and Active Pillar State (overall working of everything together) **[30 Marks]**	• Navigation is broken, missing, or causes the application backend to crash during page switching. • Application does not include Part 1 works	• Navigation menu is present, but switching views drops state or introduces noticeable UI freezing. • Some of Part 1 work is included or does not work perfectly	• Menu functions well, providing smooth transitions between Part 1 and Part 2 views with minor state-tracking quirks.	• Flawless, asynchronous frontend routing that seamlessly preserves application states while shifting between telemetry modules.	
	0 - 8 Mark	**9 - 16 Marks**	**17 – 20 Marks**	**21 - 30 Marks**	
Advanced Collections: Stacks and Queues **[15 Marks]**	• FIFO queues, priority queues, or command stacks are completely missing.	• Structures are present but misconfigured; high-priority alerts do not bypass the standard stream, or stack-undo crashes.	• Solid use of Queue<T> for data ingestion, priority structures for critical alerts, or Stack<T> for manual command undo history.	• Exceptional streaming logic; priority queues ingest critical alerts seamlessly without latency, and the stack-based command history functions flawlessly.	
	0 – 4 Marks	**5 - 10 Marks**	**11 - 14 Marks**	**15 Marks**	

Lookup Structures: Dictionaries and Key-Value Maps **[15 Marks]**	• Fast lookup maps are absent; application relies on slow, nested loops to search for active devices.	• Dictionaries are used, but keys are poorly designed, causing hash collisions, or timelines fail to sort correctly.	• Dictionary<TKey, TValue> provides O(1) device lookups via MAC addresses; or sorted dictionaries maintain order on timeline graphs.	• Seamless integration; device caching is instantaneous via dictionaries, and timelines load instantly via highly optimised sorted structures.	
	0 – 4 Marks	**5 - 10 Marks**	**11 - 14 Marks**	**15 Marks**	
Unique Collections: Sets **[10 Marks]**	• Sets are omitted, leading to duplicate alert processing.	• Sets are present but utilised incorrectly, or duplicate warnings leak through to the interface.	• Good use of HashSet<T> to maintain unique tracking of disconnected nodes or hardware warnings	• Unique tracking lists are perfectly optimised; the set prevents duplicate overhead, showcasing advanced algorithmic mastery	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8Marks**	**9-10 Marks**	
Predictive Action and Suggestion Engine **[30 Marks]**	• The suggestion engine is completely absent, non-functional, or omitted.	• Engine is present, but predictions appear random, disconnected from user search history, or cause performance drops.	• The system successfully tracks user query history, identifying patterns and outputting correct automated dashboard recommendations.	• Highly sophisticated analysis engine that intelligently anticipates operational actions, serving clean, real-time proactive alerts and makes use of the data structure to its fullest.	
	0 - 8 Mark	**9 - 16 Marks**	**17 – 20 Marks**	**21 - 30 Marks**	

Incorrect GitHub used or not GitHub Used -5 Marks	• No GitHub repository submitted.	• GitHub repository submitted with limited commit history and limited evidence of collaboration. • No README.md file included.	• GitHub repository submitted with good commit history (25 commits). • Evidence of collaboration (multiple contributors). • README.md file included but not formatted in Markdown.	• GitHub repository submitted with excellent commit history (25+ commits and well-structured commit messages). • Evidence of collaboration (multiple contributors). • README.md file included and perfectly formatted in Markdown.	
	-5 Marks	**-4 or -3 Marks**	**-2 or -1 Marks**	**No deduction**	
PART 2 TOTAL					**/100**
Notes to Students:					

PART 3					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Tree Structures and Historical Telemetry Queries **[30 Marks]**	• Tree structures are missing or completely unlinked from historical queries.	• Custom or built-in tree structures are present, but retrieval bugs exist when filtering specific telemetry date or time ranges.	• Trees (BST, AVL, or Red-Black structures) are well-implemented, executing timestamp-filtered queries effectively with minor inefficiencies.	• Exceptional tree structure architecture; self-balancing trees execute range queries on extensive telemetry history with lightning speed.	
	0 - 8 Mark	**9 - 16 Marks**	**17 – 20 Marks**	**21 - 30 Marks**	
Graphs, Network Traversals, and Minimum Spanning Trees **[30 Marks]**	• Graphs and routing algorithms are absent, leaving mesh network modelling entirely unsupported.	• The topology graph is constructed, but network traversals (BFS/DFS) or MST routing algorithms are present but yield inaccurate paths.	• Graphs accurately model nodes as vertices and links as edges; traversal and power-efficient routing calculations work with minor flaws.	• Flawless graph integration; Prim's or Kruskal's algorithm calculates optimal, power-efficient hops through complex mesh nodes effortlessly.	
	0 - 8 Mark	**9 - 16 Marks**	**17 – 20 Marks**	**21 - 30 Marks**	
Implementation Report: Readme and Structure Code Explanations **[10 Marks]**	• No documentation or explanation of the implementation is provided.	• Documentation is minimal; readme lacks execution steps, or code explanations fail to describe why graphs/trees were needed.	• Good readme provided; text successfully details the role of trees, graphs, and routing algorithms with a few basic code examples.	• Masterfully written readme and architectural analysis; provides rich, in-depth code examples showing exactly how performance scales.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8Marks**	**9-10 Marks**	

PART 3					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Project Completion Report: Challenges and Learnings **[10 Marks]**	• Completion report is entirely missing.	• The report is vague, providing generic discussion about problems faced and general programming skills • No link to how those skills were gained from the assignment/task	• Complete report detailing development challenges across all tasks, explaining how specific API/UI roadblocks were broken down.	• Deeply analytical engineering reflection detailing precise architectural hurdles, debugging approaches, and comprehensive conceptual gains.	
	0 – 3 Marks	**4 - 6 Marks**	**7 - 8Marks**	**9-10 Marks**	
Technology Recommendations and Future Scaling **[5 Marks]**	• No tech or architectural recommendations are provided.	• Recommendations are generic, uninspired, or completely detached from modern cloud or dev tools.	• Practical tool or cloud recommendations are made, with basic justifications centred on optimising IoT tracking ecosystems.	• Visionary recommendations detailing modern tools (e.g. cloud ingestion pipelines, container orchestration, telemetry analysis), thoroughly justified.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	
Technology Recommendations: Rationale **[5 Marks]**	• No justification or rationale is provided.	• Justifications are unclear or fail to outline the exact performance, security, or workflow improvements to the .NET ecosystem.	• Rationale is clear, directly linking tool recommendations to potential architecture benefits.	• Outstanding, highly detailed technical rationale demonstrating how the suggested tools optimise security, scalability, and performance.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	

PART 3					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Iterative Feedback Incorporation [5 Marks]	• No update ledger or modification tracking file is submitted.	• Ledger is submitted but shows minimal attempt to refine code based on previous critique.	• Complete update log tracking specific bugs squashed, and refinements applied following previous evaluations.	• A meticulous, professional engineering ledger documenting continuous code optimisation, directly demonstrating structural improvements.	
	0 Mark	**1 – 2 Marks**	**3 – 4 Marks**	**5 Marks**	
Main Menu Integration and Architecture Health [5 Marks]	• Final menu is broken, or previous modules fail to run inside the final integrated application.	• Modules are stitched together, but features from earlier parts are sluggish or throw errors when navigating.	• All 3 pillars function smoothly under a clean unified interface, running without blocking performance.	• A flawless, enterprise-grade unified interface where all 3 core modules operate beautifully, exhibiting asynchronous mastery.	
	0 Mark	**1 - 2 Marks**	**3 – 4 Marks**	**5 Marks**	

Incorrect GitHub used or not GitHub Used -5 Marks	• No GitHub repository submitted	• GitHub repository submitted with limited commit history and limited evidence of collaboration. • No README.md file included	• GitHub repository submitted with good commit history (25 commits) • Evidence of collaboration (multiple contributors). • README.md file included but not formatted in Markdown.	• GitHub repository submitted with excellent commit history (25+ commits and well-structured commit messages). • Evidence of collaboration (multiple contributors). • README.md file included and perfectly formatted in Markdown.	
	-5 Marks	**-4 or -3 Marks**	**-2 or -1 Marks**	**No deduction**	
PART 3 TOTAL					**/100**
Notes to Students:					

Notes for Marker

Assign marks according to the following levels of achievement:

- ***POOR: Not completed/not submitted.***
- ***DEVELOPING: Attempted but incorrect.***
- ***GOOD: Logic/process and coding correct.***
- ***EXCELLENT: Logic/process correct, and coding correct and efficient. Student has gone above and beyond specified requirements.***